

এখনকার দিনে অনেক সময় খুব বড় **dataset** নিয়ে কাজ করতে হয়। বিশেষ করে যদি তুমি **BERT** বা **GPT-2**-এর মতো transformer model শুরু থেকে train করতে চাও, তাহলে dataset অনেক বড় হয়।

সমস্যা হলো:

- dataset কয়েক **GB** বা শত **GB** হতে পারে
- এত বড় data একবারে **RAM**-এ তোলা কঠিন
- laptop বা desktop hangও করতে পারে

উদাহরণ হিসেবে, **GPT-2** train করার জন্য যে **WebText** dataset ব্যবহার করা হয়েছিল, সেটাতে ছিল:

- ৮ মিলিয়নের বেশি **document**
- প্রায় **40 GB text**

এত বড় data যদি laptop-এর RAM-এ একবারে load করতে যাও, তাহলে সেটা বড় সমস্যা হতে পারে।

Datasets কীভাবে সাহায্য করে?

Datasets লাইব্রেরি এই সমস্যা কমায় দুইভাবে:

1) Memory mapping ব্যবহার করে

মানে dataset-কে পুরোটা RAM-এ তোলার দরকার হয় না।

2) Streaming ব্যবহার করে

মানে data দরকার হলে তখন তখন আনা যায়, পুরো dataset আগে download করতে হয় না।

এই অংশে কোন **dataset** নিয়ে কথা বলছে?

এখানে তারা **The Pile** নামে একটা বিশাল dataset-এর কথা বলছে।

The Pile কী?

The Pile হলো একটি বড় **English text corpus**

যেটা **EleutherAI** বড় language model train করার জন্য বানিয়েছে।

এতে অনেক ধরনের data আছে, যেমন:

- scientific article
- GitHub code
- filtered web text

পুরো dataset-এর size প্রায় **825 GB**।

এটা খুবই বড়।

তারা কোন **subset** দেখিয়েছে?

তারা **PubMed Abstracts** dataset দেখিয়েছে।

এটা হলো:

- PubMed-এর biomedical publication-এর abstract
- প্রায় **15 million** paper-এর abstract

অর্থাৎ medical research paper-এর summary-type text।

dataset load করার আগে **zstandard** install কেন?

```
!pip install zstandard
```

কারণ file-টা compressed ছিল **.zst** format-এ।

এটা খুলতে **zstandard** দরকার।

dataset কীভাবে load করেছে?

```
from datasets import load_dataset
```

```
data_files =
```

```
"https://the-eye.eu/public/AI/pile_preliminary_components/PUBMED_title_abstracts_2019_baseline.jsonl.zst"
```

```
pubmed_dataset = load_dataset("json", data_files=data_files, split="train")
```

এখানে কী হচ্ছে:

- file remote URL থেকে আনা হচ্ছে
- file type হলো JSON
- dataset-এর **train** split load করা হচ্ছে

output কী দেখালো?

```
Dataset({  
  features: ['meta', 'text'],  
  num_rows: 15518009  
})
```

এর মানে:

- 2টা column আছে:
 - **meta**
 - **text**
- মোট row আছে:
 - **15,518,009**

মানে অনেক বড় dataset।

প্রথম **example**-এ কী আছে?

```
pubmed_dataset[0]
```

এতে দেখা যাচ্ছে:

- **meta**-র ভিতরে article-এর তথ্য আছে
 - যেমন **pmid**
 - language
- **text**-এ article-এর abstract আছে

অর্থাৎ প্রতিটি row-তে একটা medical paper-এর abstract রাখা আছে।

এখন আসল **interesting part: RAM** এত কম কেন লাগল?

তারা **psutil** দিয়ে RAM usage check করেছে।

```
!pip install psutil
```

তারপর:

```
import psutil  
print(f"RAM used: {psutil.Process().memory_info().rss / (1024 * 1024):.2f} MB")
```

এখানে **rss** মানে process RAM-এ কত memory ব্যবহার করেছে।

Output ছিল প্রায়:

RAM used: 5678.33 MB

অর্থাৎ প্রায় **5.7 GB RAM** ব্যবহার হয়েছে।

কিন্তু **dataset size** তো অনেক বড় ছিল

তারপর তারা dataset-এর actual size দেখেছে:

```
print(f"Dataset size in bytes: {pubmed_dataset.dataset_size}")  
size_gb = pubmed_dataset.dataset_size / (1024**3)  
print(f"Dataset size (cache file) : {size_gb:.2f} GB")
```

Output:

- dataset size প্রায় **19.54 GB**

অর্থাৎ:

- dataset disk-এ প্রায় **20 GB**
- কিন্তু RAM লাগছে প্রায় **5.7 GB**

এটাই **Datasets**-এর বড় সুবিধা।

এটা কীভাবে সম্ভব?

এটার কারণ হলো **memory mapping**।

Memory mapping কী?

সহজ ভাষায়:

dataset পুরোটা RAM-এ তোলার বদলে,
file-টা disk-এ থাকে, আর program প্রয়োজনমতো ছোট ছোট অংশ access করে।

মানে:

- পুরো বই মাথায় রাখতে হচ্ছে না
- দরকার হলে শুধু ওই page খুলে পড়ছ

এই কারণে RAM অনেক কম লাগে।

Pandas-এর সাথে পার্থক্য কোথায়?

Pandas-এ সাধারণত বড় dataset load করতে গেলে অনেক RAM লাগে।
অনেক সময় dataset size-এর ৫ থেকে ১০ গুণ RAM দরকার হতে পারে।

কিন্তু **Datasets** memory-mapped file ব্যবহার করে, তাই:

- সব data RAM-এ একসাথে তোলে না
- প্রয়োজনমতো access করে
- RAM usage কম হয়

Apache Arrow-এর ভূমিকা কী?

এই speed আর efficiency-এর পেছনে বড় কারণ হলো:

- **Apache Arrow**
- **pyarrow**

এগুলো data-কে এমন format-এ রাখে যাতে:

- খুব দ্রুত পড়া যায়
- কম memory লাগে
- processing fast হয়

আরেকটা বড় সুবিধা: **parallel processing**

Memory-mapped file অনেক process share করতে পারে।

এর মানে:

- `Dataset.map()` parallel ভাবে চালানো যায়
- data copy করতে হয় না
- কাজ আরও fast হয়

তারা **speed test**-ও করেছে

তারা পুরো dataset batch batch করে iterate করেছে:

```
import timeit
```

```
code_snippet = """batch_size = 1000
```

```
for idx in range(0, len(pubmed_dataset), batch_size):
    __ = pubmed_dataset[idx:idx + batch_size]
    """
```

এখানে:

- একবারে 1000 row করে নিচ্ছে
- এভাবে পুরো dataset ঘুরে দেখছে

তারপর `timeit` দিয়ে সময় মাপেছে।

Output:

Iterated over 15518009 examples (about 19.5 GB) in 64.2s, i.e. 0.304 GB/s

এর মানে:

- 15 million-এর বেশি example পড়তে লেগেছে প্রায় **64.2 second**
- speed ছিল প্রায় **0.304 GB** প্রতি **second**

এটা বেশ ভালো speed।

তারপরও সমস্যা কোথায় থাকতে পারে?

ধরো dataset এত বড় যে:

- RAM-এ ধরবে না
- hard drive-এই জায়গা নেই

যেমন পুরো **Pile** dataset download করতে গেলে **825 GB** free space লাগবে।

এটা বেশিরভাগ laptop-এ সম্ভব না।

এই সমস্যা সমাধান: Streaming

এই জায়গায় **Datasets** streaming feature দেয়।

Streaming dataset মানে কী?

সাধারণভাবে যখন তুমি dataset load করো, তখন library dataset-টা ব্যবহার করার জন্য local system-এ প্রস্তুত করে নেয়।

কিন্তু dataset যদি খুব বড় হয়, তাহলে:

- download করতে সময় লাগে
- disk space বেশি লাগে
- পুরো dataset রাখা কঠিন হয়

এই সমস্যা সমাধানের জন্য **Datasets**-এ streaming আছে।

Streaming-এর idea

পুরো dataset আগে download না করে, যখন যতটুকু **data** দরকার, তখন ততটুকু আনা।

একদম video streaming-এর মতো:

- পুরো movie আগে download করো না
- যতটুকু play হচ্ছে, ততটুকু আসে

dataset-এর ক্ষেত্রেও একই ব্যাপার।

1) Streaming চালু করতে কী করতে হয়?

শুধু `load_dataset()`-এ `streaming=True` দিতে হয়।

```
pubmed_dataset_streamed = load_dataset(
```

```
"json",  
data_files=data_files,  
split="train",  
streaming=True  
)
```

এখানে dataset stream mode-এ load হচ্ছে।

2) এতে কী ধরনের **object** পাওয়া যায়?

আগে আমরা **Dataset** object পেতাম।

কিন্তু **streaming=True** দিলে **IterableDataset** পাওয়া যায়।

IterableDataset মানে কী?

এর মানে dataset-টাকে একটার পর একটা **iterate** করে access করতে হবে।

মানে:

- normal indexing-এর মতো সবসময় সহজে **dataset[0]** করা যাবে না
- বরং loop বা iterator দিয়ে data নিতে হবে

3) প্রথম **element** কিভাবে বের করে?

```
next(iter(pubmed_dataset_streamed))
```

এখানে:

- **iter(...)** dataset-টাকে iterator বানাচ্ছে
- **next(...)** প্রথম example দিচ্ছে

অর্থাৎ streamed dataset থেকে প্রথম row নেওয়া হচ্ছে।

4) Streaming dataset-এ **processing** করা যায়?

হ্যাঁ, যায়।

`IterableDataset.map()` ব্যবহার করে **on the fly processing** করা যায়।

উদাহরণ হিসেবে tokenization করেছে:

```
from transformers import AutoTokenizer
```

```
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")
tokenized_dataset = pubmed_dataset_streamed.map(lambda x: tokenizer(x["text"]))
next(iter(tokenized_dataset))
```

এখানে কী হচ্ছে:

- `text` field নেওয়া হচ্ছে
- tokenizer দিয়ে token বানানো হচ্ছে
- output হিসেবে `input_ids`, `attention_mask` আসছে

সহজভাবে

dataset আসছে, আর আসার সাথে সাথে process হচ্ছে।

মানে আগে সব data tokenize করে save করতে হচ্ছে না।

5) `batched=True` কেন useful?

Streaming-এ tokenization fast করতে `batched=True` ব্যবহার করা যায়।

মানে:

- একবারে ১টা example process না করে
- এক batch example একসাথে process করবে

এতে speed বাড়ে।

Default batch size সাধারণত **1000**।

6) Streamed dataset shuffle করা যায়?

হ্যাঁ, যায়।

কিন্তু normal `Dataset.shuffle()` আর streamed `IterableDataset.shuffle()` একরকম না।

```
shuffled_dataset = pubmed_dataset_streamed.shuffle(buffer_size=10_000, seed=42)
```

এখানে পুরো dataset একসাথে shuffle হচ্ছে না।

শুধু একটা **buffer**-এর ভেতরের data shuffle হচ্ছে।

Buffer মানে কী?

এখানে `buffer_size=10000` মানে:

- একসাথে ১০,০০০ example memory-তে রাখা হবে
- ওই ১০,০০০-এর মধ্যে random shuffle হবে

তারপর:

- একটা example বের হলে
- তার জায়গায় dataset-এর পরের example ঢুকে যাবে

সহজভাবে

পুরো নদী না নেড়ে, এক বালতি পানির ভেতরে নেড়েচেড়ে random করা হচ্ছে।

7) `take()` কী করে?

```
dataset_head = pubmed_dataset_streamed.take(5)  
list(dataset_head)
```

এটা dataset-এর প্রথম ৫টা **example** নেয়।

`take(5)` মানে প্রথম ৫টা row দরকার।

তারপর `list(...)` দিলে সবগুলো একসাথে দেখা যায়।

8) `skip()` কী করে?

`skip(n)` মানে প্রথম `n` টা example বাদ দিয়ে বাকিগুলো নেয়।

উদাহরণ:

```
train_dataset = shuffled_dataset.skip(1000)
validation_dataset = shuffled_dataset.take(1000)
```

এখানে:

- প্রথম 1000 example → validation
- বাকি সব → training

সহজভাবে

প্রথম 1000 টা আলাদা রাখলাম validation-এর জন্য, তারপর বাকিটা training।

9) কেন **take()** আর **skip()** দরকার?

কারণ streamed dataset-এ normal **select()** সবসময় convenient না।

তাই:

- **take()** → সামনে থেকে কিছু নাও
- **skip()** → সামনে থেকে কিছু বাদ দাও

এগুলো দিয়ে সহজে split বানানো যায়।

10) একাধিক **streamed dataset** একসাথে মিলানো যায়?

হ্যাঁ, যায়।

এর জন্য **interleave_datasets()** ব্যবহার করা হয়।

এটা খুব useful যখন তুমি অনেক বড় dataset মিশিয়ে একটা বড় **corpus** বানাতে চাও।

11) FreeLaw dataset কী?

উদাহরণ হিসেবে আরেকটা বড় dataset নেওয়া হয়েছে:

FreeLaw Opinions

এটা:

- US courts-এর legal opinion
- প্রায় **51 GB** size

এটাও stream mode-এ load করেছে:

```
law_dataset_streamed = load_dataset(  
    "json",
```

```
    data_files="https://the-eye.eu/public/AI/pile_preliminary_components/FreeLaw_Opinions.jsonl.z  
st",  
    split="train",  
    streaming=True,  
)
```

অর্থাৎ:

- dataset বড়
- তবুও stream mode-এ সহজে access করা যাচ্ছে
- RAM-এ চাপ কম

12) `interleave_datasets()` কী করে?

```
from datasets import interleave_datasets
```

```
combined_dataset = interleave_datasets([  
    pubmed_dataset_streamed,  
    law_dataset_streamed  
)
```

এটা দুইটা streamed dataset একসাথে মিলিয়ে নতুন একটা dataset বানায়।

কীভাবে মিলায়?

একটার পর একটা **alternating** করে।

মানে:

- ১ম example → PubMed থেকে
- ২য় example → FreeLaw থেকে
- ৩য় example → PubMed থেকে
- ৪র্থ example → FreeLaw থেকে

এভাবে চলতে থাকে।

সহজভাবে

দুইটা line থেকে পালা করে একজন করে ভেতরে ঢুকছে।

13) `islice()` কেন ব্যবহার করেছে?

```
from itertools import islice
```

```
list(islice(combined_dataset, 2))
```

এটা combined dataset-এর প্রথম ২টা **example** দেখার জন্য।

কারণ streamed dataset iterator-based, তাই `islice()` দিয়ে কয়েকটা sample দেখা convenient।

14) পুরো Pile-ও stream করা যায়?

হ্যাঁ।

তারা পুরো **825 GB Pile** dataset stream করার example দিয়েছে।

```
base_url = "https://the-eye.eu/public/AI/pile/"
data_files = {
    "train": [base_url + "train/" + f"{idx:02d}.jsonl.zst" for idx in range(30)],
    "validation": base_url + "val.jsonl.zst",
    "test": base_url + "test.jsonl.zst",
}

pile_dataset = load_dataset("json", data_files=data_files, streaming=True)
```

এখানে:

- train-এর জন্য অনেকগুলো file দেওয়া হয়েছে

- validation আর test-এর জন্য আলাদা file
- সব stream mode-এ load হচ্ছে

অর্থাৎ পুরো 825 GB dataset একবারে নামাতে হচ্ছে না।

15) এর সবচেয়ে বড় সুবিধা কী?

Streaming-এর বড় সুবিধাগুলো হলো:

কম **disk space** লাগে

পুরো dataset save করে রাখতে হয় না।

কম **RAM** লাগে

দরকারমতো data আসে।

huge dataset handle করা যায়

নিজের laptop দিয়েও বড় corpus ব্যবহার করা সম্ভব।

training-এর সময় useful

বিশেষ করে যখন dataset অনেক বড়।

16) Normal Dataset আর Streaming Dataset-এর পার্থক্য

বিষয়	Normal Dataset	Streaming Dataset
Object type	<code>Dataset</code>	<code>IterableDataset</code>
Access	indexing / slicing	iteration

পুরো data download	সাধারণত লাগে	লাগে না
huge dataset-এর জন্য	সীমাবদ্ধ হতে পারে	খুব useful
shuffle	full shuffle সম্ভব	buffer-based shuffle

একদম সহজ ভাষায় সারাংশ

streaming=True

পুরো dataset আগে download না করে, দরকারমতো data আনে।

IterableDataset

এটাকে loop বা iterator দিয়ে পড়তে হয়।

map()

data আসার সাথে সাথে process করা যায়।

shuffle(buffer_size=...)

পুরো dataset না, শুধু buffer-এর মধ্যে shuffle হয়।

take(n)

প্রথম n টা example নেয়।

skip(n)

প্রথম n টা বাদ দেয়।

interleave_datasets()

একাধিক dataset পালা করে মিলিয়ে দেয়।